The Euclidean algorithm is not optimal among uniform processes

Jason Davies*

October 23, 2026

Abstract

We construct remainder-only gcd derivations of length $O(\log a/\log\log a)$, uniformly for $a > b \geq 1$, by reusing a stored Euclidean prefix. These yield a single uniform process with the same bound, refuting Moschovakis's conjectured logarithmic bounds on intrinsic complexity. The Main Conjecture for algorithms remains open. We also prove an $\Omega(\log\log a)$ lower bound for almost all coprime pairs and give a family of finite nondeterministic gcd programs with arbitrarily small logarithmic cost coefficients. The arithmetic and program bounds are formalized in Lean 4.

1 Introduction

Van den Dries and Moschovakis asked whether the Euclidean algorithm is optimal among its peers, the algorithms that compute the gcd from the remainder function [8, p. 390]. Moschovakis's Main Conjecture asserts that for every algorithm α from $\mathrm{rem}, \mathrm{eq}_0$ computing $\gcd(x, y)$ there is an $r > 0$ such that α makes more than $r \log x$ remainder calls on infinitely many pairs $x > y \geq 1$ [6, p. 2]; here eq_0 is the zero test. Throughout, "the book" means Moschovakis's monograph [6], and logarithms are to base 2.

Derivations. A *derivation* from a, b is a finite list that starts with a, b and in which each further entry is $\mathrm{rem}(x, y)$ for earlier entries x, y with $y > 0$. Each further entry is one *instruction*, that is, one remainder call. Entries already in the list are *available* and may be reused; no constants and no other operations are supplied. Let $G(a, b)$ be the least number of instructions in a derivation containing $\gcd(a, b)$, and let $D(a, b) = G(a, b)$ for coprime inputs. Thus $G(a, 0) = 0$ and $D(a, 1) = 0$. Arithmetic used in the proofs, such as subtraction, sorting or comparison, only selects instructions; it is never itself an instruction.

Theorem 1. *Uniformly for integers $a > b \geq 1$,*

$$G(a, b) = O\left(\frac{\log a}{\log\log a}\right).$$

In particular this bound holds for $D(a, b)$ when $\gcd(a, b) = 1$.

Idea of the proof. For sufficiently large n, let $a > b$ be coprime with $a \leq N = 2^n$. Store a Euclidean prefix $r_0 = a, r_1 = b, \ldots, r_h$, where $h = \lceil\sqrt{n}\rceil$. If the prefix reaches a positive remainder at most $N^{3/4}$, take one further remainder and recurse on the resulting smaller pair. Otherwise its consecutive pairs stay distinct modulo every $m > N^{1/3}$ (Lemma 3). There are therefore at least

*jason@jasondavies.com

$\sqrt{h}$ distinct prefix residues modulo m. A close pair of residues lets three remainder calls produce a positive value below $m/(\sqrt{h}-1)$ (Lemma 2). Reusing the same prefix, we repeatedly shrink the modulus from r_h to at most $N^{1/3}$ in $O(n/\log n)$ calls. Two recursive gcd derivations on inputs at most $N^{1/3}$ finish. The resulting recurrence (5) gives $T(n) = O(n/\log n)$, where $T(n)$ is the worst derivation cost for inputs at most 2^n. Tuning the prefix length sharpens the upper coefficient to $6 + o(1)$ (Corollary 4).

Consequences and further results. Uniform processes are the book's abstract model of computation from given primitives; every deterministic or nondeterministic recursive program induces one [6, §4A, Proposition 4A.1]. Section 5 converts the derivations into small certificates and uses the book's best-uniform-process theorem to obtain a single process with the same asymptotic bound on every input. This refutes Conjecture 3.4 of [5] and version (4F-5) of the Main Conjecture, and limits what the homomorphism method can prove. The Main Conjecture for algorithms remains open.

We also prove that $D(a,b) \geq (1-o(1)) \log\log N$ for all but $o(N^2)$ coprime pairs $2 \leq b < a \leq N$ (Proposition 9). Finally, fixing the prefix length gives finite nondeterministic programs with arbitrarily small logarithmic coefficients, uniformly over all inputs (Theorem 10). Their program size grows as the requested coefficient decreases.

Related work. The known intrinsic lower bound is $\Omega(\log\log a)$ on special families of coprime pairs [8, 9], [6, Theorem 6C.5]. Pratt's nondeterministic algorithm, from his unpublished manuscript and presented in the book [6, Theorem 2E.2 and Problems x2E.17–x2E.19], matches this order on successive Fibonacci numbers [6, Corollary 6C.7]. Our almost-all bound establishes the same order on a set of full density among coprime pairs; our program family gives an all-input upper bound.

In a different model, Bshouty obtained the optimal bound $\Theta(\log N/\log\log N)$ for gcd on inputs in $[0, N]$ using a random access machine with linear operations, shifts and indirect addressing. His construction reads precomputed blocks of the binary gcd algorithm [7], [3, §4.5.2] from a table [1, Theorems i and ii]. Here the saving comes from reusing a Euclidean prefix with remainder as the only instruction.

Organization. Sections 2–4 prove the upper bound. Section 5 gives its certificate and uniform-process consequences. Sections 6 and 7 prove the almost-all bound and the program-family theorem. Sections 8 and 9 discuss open problems and computational checks; Appendix A records the formalization.

2 Normalization

Write $g = \gcd(a,b) > 0$. For positive y,

$$\operatorname{rem}(gx, gy) = g \operatorname{rem}(x, y).$$

Every generated value is divisible by g. Consequently scaling a derivation, or dividing all its values by g, preserves its length and the validity of every instruction. Hence

$$G(a,b) = D(a/g, b/g). \tag{1}$$

Define

$$T(n) = \max_{0 \leq b \leq a \leq 2^n,\ a>0} G(a,b).$$

Each $G(a,b)$ is defined, since the Euclidean remainders form a derivation that contains $\gcd(a,b)$, so $T(n)$ is a finite maximum. By (1) it suffices to bound D; the cases $a=b$, $b=0$ and $b=1$ need no instructions.

3 Three-call contractions

Lemma 2. *If $m>0$ is available and a finite collection S of available values occupies $t\ge 2$ residue classes modulo m, at most three instructions generate w with*

$$0<w<\frac{m}{t-1}.$$

Proof. Sort the distinct residues. Some adjacent pair $u<v$ has gap $\delta=v-u<m/(t-1)$, because the sum of the $t-1$ adjacent gaps is less than m. Each selected residue is $\operatorname{rem}(x,m)$ for some $x\in S$, so it costs one instruction. If $u=0$, use v. If $0<u\le\delta$, use u. Otherwise $0<\delta<u$ and $v=u+\delta$, so $\operatorname{rem}(v,u)=\delta$. This uses at most three instructions. □

Lemma 3 (Euclidean-prefix lemma)**.** *Let $r_0=a>r_1=b>\cdots>r_h>0$ be a Euclidean prefix of a coprime pair, with $r_{i-1}=q_ir_i+r_{i+1}$ and $q_i\ge 1$ for $1\le i<h$. For every integer $m>2a/r_h$, the h pairs*

$$(r_i \bmod m, r_{i+1} \bmod m), \qquad 0\le i<h,$$

are distinct. The prefix therefore occupies t residue classes with $t^2\ge h$.

Proof. Suppose the pairs at $i<j$ coincide. Set

$$M=\prod_{\ell=i+1}^{j}\begin{pmatrix}q_\ell & 1\\ 1 & 0\end{pmatrix}=\begin{pmatrix}A & B\\ C & E\end{pmatrix}, \qquad \begin{pmatrix}r_i\\ r_{i+1}\end{pmatrix}=M\begin{pmatrix}r_j\\ r_{j+1}\end{pmatrix}.$$

Right multiplication by $\left(\begin{smallmatrix}q & 1\\ 1 & 0\end{smallmatrix}\right)$ sends (A,B,C,E) to $(qA+B,\ A,\ qC+E,\ C)$, starting from $(q,1,1,0)$ for a single factor. By induction, all entries are nonnegative, $A\ge B\ge E$, $A\ge C\ge 1$, $B\ge 1$, and $\det M=(-1)^{j-i}$. Also $Ar_j\le r_i\le a$ and $r_j>r_h$ (as $j<h$), whence $A<a/r_h$. Write

$$\Delta=\det(M-I)=1+\det M-A-E.$$

For an odd number of factors, $\Delta=-A-E<0$. By the same recursion, a product of at least two factors has $A\ge 2$ and $E\ge 1$; thus for an even positive number of factors, $\Delta=2-A-E<0$. In either case,

$$0<|\Delta|\le A+E\le 2A\le 2a/r_h<m.$$

The presumed congruence gives $(M-I)(r_j,r_{j+1})^{\mathsf T}\equiv 0 \pmod m$. Multiplication by the adjugate shows that m divides both Δr_j and Δr_{j+1}. Consecutive remainders are coprime, so Bézout's identity implies $m\mid\Delta$, a contradiction. Finally, t individual residue classes give at most t^2 ordered pairs. □

Thus a stored prefix can be reused for every available modulus $m>2a/r_h$. When $\sqrt{h}>1$, Lemma 2 applied to the prefix gives three instructions that generate

$$0<w<\frac{m}{\sqrt{h}-1}. \tag{2}$$

We call these three instructions a *contraction* of m, and w the new modulus. The universal quantifier over m is essential: the modulus generated by a contraction is not predetermined. In Lemma 3 the prefix length h is generic; Section 4 chooses $h=\lceil\sqrt{n}\rceil$, while Corollary 4 and Section 7 choose $h=k^2+1$.

4 Construction and recurrence

Fix an integer $n > 16$ and put $N = 2^n$ and $h = \lceil \sqrt{n} \rceil > 4$. After normalization, assume $a > b > 1$ are coprime and $a \le N$. We use $N^{3/4}$ as the *early threshold* and $N^{1/3}$ as the *target modulus*. With thresholds N^α, N^β, the prefix lemma applies above the target when $\alpha + \beta > 1$, and the two final recursive problems together have smaller bit size when $2\beta < 1$. Also $\alpha < 1$ makes the early recursive problem smaller. The choices $\alpha = 3/4$, $\beta = 1/3$ satisfy all three conditions.

Phase (i): the prefix. Generate the Euclidean remainders $r_2, r_3, \ldots$, one instruction each, and stop as soon as the most recent value is positive and at most $N^{3/4}$, or once r_h is available. The test is also applied to the input $r_1 = b$, so if $b \le N^{3/4}$ no instruction is used. This preliminary work costs at most h instructions. Since the inputs are coprime, Euclid's algorithm reaches the positive remainder 1 before it can reach zero, so the rule stops before a zero remainder appears.

If a positive remainder $c \le N^{3/4}$ is encountered, generate the next remainder $s < c$ with one more instruction. The pair (c, s) is coprime. If $s = 0$, then $c = 1$ is already available; otherwise both inputs to the remaining derivation are at most $2^{\lceil 3n/4 \rceil}$. This gives the bound

$$h + 1 + T(\lceil 3n/4 \rceil). \tag{3}$$

Phase (ii): contractions. Otherwise $r_h > N^{3/4}$, so $2a/r_h < 2N^{1/4}$. For $n > 12$, $N^{1/3} > 2N^{1/4}$, and Lemma 3 applies to every available $m > N^{1/3}$. Starting at $m = r_h$, repeatedly replace m by the value w of (2) until $0 < m \le N^{1/3}$. Set

$$\kappa_n = \sqrt{h} - 1 > 1, \qquad J_n = \left\lceil \frac{2n/3}{\log \kappa_n} \right\rceil.$$

There are at most J_n contractions: if J_n have occurred then $m < N/\kappa_n^{J_n} \le N^{1/3}$. Since $\log \kappa_n = \frac{1}{4} \log n + O(1)$, their cost is $3J_n = O(n/\log n)$.

Phase (iii): two recursive derivations. The small m need not be coprime to either input. Generate $u = \mathrm{rem}(a, m)$ and $v = \mathrm{rem}(b, m)$, then concatenate derivations of $d = \gcd(m, u)$ and $e = \gcd(d, v)$; no gcd-oracle instruction is used. Since $m \le 2^{\lceil n/3 \rceil}$ and G is symmetric, each has a remainder-only derivation of length at most $T(\lceil n/3 \rceil)$. Each sub-derivation starts from values already in the list (m, u, and then d, v), so it can be appended unchanged. The gcd identities give

$$e = \gcd(\gcd(m, a \bmod m), b \bmod m) = \gcd(m, \gcd(a, b)) = 1.$$

The cost in this case is at most

$$h + 3J_n + 2 + 2T(\lceil n/3 \rceil). \tag{4}$$

The recurrence. The two case estimates (3) and (4) and normalization imply

$$T(n) \le \max\{T(\lceil 3n/4 \rceil) + O(\sqrt{n}),\ 2T(\lceil n/3 \rceil) + O(n/\log n)\}. \tag{5}$$

All implied constants are independent of the inputs.

Proof of Theorem 1. For $n \ge 2$ let $f(n) = n/\log n$. The ratios

$$\frac{f(\lceil 3n/4 \rceil)}{f(n)} \longrightarrow \frac{3}{4}, \qquad \frac{2f(\lceil n/3 \rceil)}{f(n)} \longrightarrow \frac{2}{3}$$

show that, beyond a fixed threshold, $f(\lceil 3n/4\rceil) \le \frac{7}{8} f(n)$ and $2f(\lceil n/3\rceil) \le \frac{5}{6} f(n)$. Both overheads in (5) are at most $c_0 f(n)$ for a fixed c_0. Choose the induction threshold above 16, where the recurrence holds, and large enough that both recursive indices are at least 2 and strictly less than n. Choose $C \ge 8c_0$, increased to cover the finitely many values below the threshold. Strong induction then gives $T(n) \le Cf(n)$: the two bounds are $(7C/8+c_0)f(n)$ and $(5C/6+c_0)f(n)$, both at most $Cf(n)$. Finally take $n = \lceil \log a \rceil$. This also gives $G(a,b) = o(\log a)$ uniformly as $a \to \infty$. □

4.1 The leading coefficient

The same construction allows a more precise bound without changing either arithmetic lemma.

Corollary 4. *For every $\varepsilon > 0$, there is an a_0 such that for every $a \ge a_0$ and $0 \le b \le a$,*

$$G(a,b) \le (6+\varepsilon)\frac{\log a}{\log\log a}.$$

Equivalently, the uniform upper bound has coefficient $6 + o(1)$.

Proof. Fix $0 < c < 1/2$ and $0 < \beta < 1/2$, and put $\alpha = 1 - \beta/2$. For sufficiently large n, choose the prefix length h afresh, together with auxiliary parameters, as

$$p = \lfloor c \log n \rfloor, \qquad k = 2^p, \qquad h = k^2 + 1, \qquad J = \lceil n/p \rceil.$$

Use N^α as the early threshold and N^β as the target modulus, where $N = 2^n$. In the late case $r_h > N^\alpha$, so $2a/r_h < 2N^{1-\alpha} = 2N^{\beta/2} < N^\beta$ for sufficiently large n, and the prefix lemma applies to every modulus above N^β. Since $k^2 < h$, the prefix occupies at least $k+1$ residue classes, so each contraction reduces the modulus by a factor greater than k. Moreover $k^J \ge N$, so J contractions suffice. Writing $f(n) = n/\log n$, we have

$$h = O(n^{2c}) = o(f(n)), \qquad 3J = \left(\frac{3}{c} + o(1)\right) f(n).$$

The two branches therefore give

$$T(n) \le \max\left\{ T(\lceil \alpha n \rceil) + o(f(n)), \quad 2T(\lceil \beta n \rceil) + \left(\frac{3}{c} + o(1)\right) f(n) \right\}.$$

Theorem 1 implies that $L = \limsup_{n\to\infty} T(n)/f(n)$ is finite and nonnegative. For each fixed $q > 0$, the ratio $f(\lceil qn \rceil)/f(n)$ tends to q. Taking upper limits in the recurrence yields

$$L \le \max\{\alpha L,\ 2\beta L + 3/c\}.$$

Since $\alpha < 1$ and $L \ge 0$, this implies $(1-2\beta)L \le 3/c$. First let $\beta \downarrow 0$ and then $c \uparrow 1/2$ to obtain $L \le 6$. Finally, $n = \lceil \log a \rceil$ satisfies $f(n) \sim \log a / \log\log a$, proving the assertion. All upper limits concern the worst-case function T, so the resulting threshold is independent of b. □

No optimality of the coefficient 6 is asserted. The parameters of Section 4 suffice for the simpler Big-O proof above.

5 Certificates and uniform processes

Structures and certificates. Write tt, ff for the truth values and eq_c for the test $x = c$. We use the book's partial remainder operation, defined only when its second argument is nonzero, in the structures

$$\mathcal{N}_0 = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0), \qquad \mathcal{N}_\varepsilon = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0, \mathrm{eq}_1).$$

The latter is the book's Euclidean structure $\mathbf{N}_\varepsilon$ [6, (1C-10), p. 24]; our $\mathcal{N}_0$ should not be confused with the book's $\mathbf{N}_0$, which has no remainder operation [6, p. 171]. Coprimeness is the relation $x \perp\!\!\!\perp y \iff x, y \geq 1 \wedge \gcd(x, y) = 1$ [6, §1A].

A derivation records finitely many remainder equations. These form a partial structure generated by the inputs; repeated equations can be discarded. A *certificate* for f at $\vec{x}$ in a structure $\mathcal{A}$ is such a finite generated partial substructure, possibly with test facts, on which every homomorphism into $\mathcal{A}$ respects f at $\vec{x}$. In other words, any assignment of values preserving the recorded operations and tests must preserve the answer. The *intrinsic remainder-call complexity* $\mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, f, \vec{x})$ is the least number of remainder facts in a certificate [6, §§1D, 4C–4E].

Proposition 5. *For coprime $a, b \geq 1$, $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) = D(a, b)$. For $(a, b) \neq (0, 0)$,*

$$G(a, b) \leq \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_0, \gcd, a, b) \leq G(a, b) + 2,$$

and the same bounds hold for $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd, a, b)$.

Proof. Write $\gcd{=}1$ for the relation $\gcd(x, y) = 1$ on all of $\mathbb{N}^2$ (unlike $\perp\!\!\!\perp$, it holds at $(1, 0)$). A derivation of 1 together with $\mathrm{eq}_1(1) = \mathrm{tt}$ certifies the relation: every common divisor of the images of the inputs divides every transported generated value, hence divides 1. Conversely, without a positive eq_1 fact the map $x \mapsto 2x$ preserves every recorded remainder and zero-test fact, as well as every negative one-test fact. It cannot preserve coprimeness. A certificate therefore contains a positive eq_1 fact, hence the value 1. Enumerating a finite generated diagram in generation order gives a derivation using no more calls than the number of remainder facts. Thus the least remainder count of a certificate for $\gcd(a, b) = 1$ is $D(a, b)$ on coprime inputs. For the book's relation $\perp\!\!\!\perp$, add $\mathrm{eq}_0(a) = \mathrm{ff}$ and $\mathrm{eq}_0(b) = \mathrm{ff}$ to ensure positive input images. Conversely, on positive coprime inputs every certificate for $\perp\!\!\!\perp$ certifies $\gcd(a, b) = 1$. These extra tests cost no remainder facts, proving the first assertion.

For gcd as a function, the certificate must reach its output g, giving a lower bound of $G(a, b)$. For an upper bound, append

$$\mathrm{rem}(a, g) = z, \qquad \mathrm{rem}(b, g) = z, \qquad \mathrm{eq}_0(z) = \mathrm{tt},$$

where the generated z is zero. Every homomorphism π has $\pi(z) = 0$ and $\pi(g) > 0$ (by definedness of either appended remainder). Thus $\pi(g)$ divides both input images. Conversely every common divisor of those images divides $\pi(g)$ by induction over the derivation. Therefore, for $(a, b) \neq (0, 0)$,

$$G(a, b) \leq \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_0, \gcd, a, b) \leq G(a, b) + 2.$$

The lower-bound argument applies to every gcd certificate, eq_1 facts included, and allowing eq_1 facts can only lower the count, so the upper bound transfers to $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd, a, b)$. □

Remark 6. At $(0, 0)$ the output is an input and the empty certificate suffices. If the gcd is taken as a function on positive inputs only, the facts $\mathrm{eq}_0(a) = \mathrm{ff}$ and $\mathrm{eq}_0(b) = \mathrm{ff}$ may be appended. For a noncoprime input, adding $\mathrm{eq}_1(g) = \mathrm{ff}$ to the gcd certificate certifies noncoprimeness, so the sublogarithmic remainder-call bound holds for both truth values of coprimeness as well as for the gcd. The overhead is a fixed additive constant and does not change the leading coefficient, so Corollary 4 holds for all three complexities of Corollary 8 below on positive inputs.

A single uniform process. Proposition 5, Remark 6, and Theorem 1 bound each intrinsic complexity by $O(\log a/\log\log a)$, uniformly for $a \geq b \geq 1$. The book's Theorem 4F.1 states that, whenever every input has a certificate, one uniform process attains the intrinsic complexity at every input [6, pp. 149–150]. It therefore gives the following consequence.

Corollary 7. *For each of gcd and coprimeness, there is a single uniform process of $\mathcal{N}_\varepsilon$ computing it on $\mathbb{N}^2$ with $O(\log a/\log\log a)$ remainder calls, uniformly for $a \geq b \geq 1$. The gcd process can also be taken over $\mathcal{N}_0$.*

Proof. The certificate bounds give the cost assertion by Theorem 4F.1. Every input has a certificate: symmetry covers reversed inputs, and a zero-test fact suffices when either input is zero. Thus the same process is defined on all of $\mathbb{N}^2$. □

The certificate bounds are formalized here; Theorem 4F.1 is cited. The corollary rules out the possibility raised after Corollary 6C.7 of the book that the Main Conjecture holds for all uniform processes of $\mathcal{N}_\varepsilon$ [6, p. 188].

The conjectures refuted. Moschovakis's 2012 notes state:

> **Conjecture 3.4** [5, p. 19]. There is a rational number $r > 0$ such that for infinitely many $a \geq b \geq 1$, $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) \geq r \log a$.

The book formulates one precise version of the Main Conjecture [6, p. 151]: there are a set $\mathcal{D} \subseteq \mathbb{N}^2$ and a $K > 0$ such that

$$(x,y) \in \mathcal{D} \implies \mathrm{calls}_{\mathrm{rem}}(\mathrm{Eucl}, x, y) \leq L(x,y) \leq K\, \mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, \gcd, x, y), \tag{4F-5}$$

where $L(x,y) = 2\log\max(x,y)$ and $\mathrm{calls}_{\mathrm{rem}}(\mathrm{Eucl}, x, y)$ counts the remainder calls of the Euclidean algorithm Eucl (the book writes ε) on (x,y). The book states (4F-5) for the Euclidean algorithm in $\mathbf{N}_\varepsilon$, so its $\mathcal{A}$ is $\mathcal{N}_\varepsilon$; we treat both $\mathcal{A} = \mathcal{N}_\varepsilon$ and $\mathcal{A} = \mathcal{N}_0$. The book adds that $\mathcal{D}$ then satisfies its conditions (4F-3) and (4F-4). Condition (4F-4) makes the intrinsic complexity unbounded on $\mathcal{D}$, so the intended $\mathcal{D}$ is infinite; for finite $\mathcal{D}$ the statement would be trivial. Dropping the first inequality and asking only for an infinite $\mathcal{D}$ gives a weaker statement, and we refute that weaker statement.

Corollary 8. *The intrinsic complexities*

$$\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b), \qquad \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_0, \gcd, a, b), \qquad \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd, a, b)$$

are $O(\log a/\log\log a)$, uniformly for $a \geq b \geq 1$. Consequently Conjecture 3.4 of [5] is false, and so is the version (4F-5) of the Main Conjecture with $\mathcal{A} = \mathcal{N}_0$ or $\mathcal{A} = \mathcal{N}_\varepsilon$: for every $r > 0$ only finitely many pairs $a \geq b \geq 1$ satisfy $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) \geq r \log a$, and for every $K > 0$ only finitely many pairs $(x,y) \in \mathbb{N}^2$ satisfy $L(x,y) \leq K\, \mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, \gcd, x, y)$.

The corollary follows from Theorem 1 and Proposition 5; for noncoprime pairs, the bound on the coprimeness complexity comes from Remark 6. For (4F-5) it covers all pairs in $\mathbb{N}^2$: the gcd is symmetric, and a pair with a zero entry is certified by the single fact $\mathrm{eq}_0(0) = \mathrm{tt}$, with no remainder facts.

The limit of the homomorphism method. The Homomorphism Test proves lower bounds on intrinsic complexity by showing that every generated diagram below the proposed cost admits a homomorphism that fails to preserve the answer [6, Lemma 4E.3]. Such bounds apply to every uniform process computing the function, hence to every recursive program [6, Proposition 4E.1, Corollary 4E.2]. By Corollary 8, any nonnegative lower bound $g(a,b)$ on one of these intrinsic complexities satisfies $g(a,b) = O(\log a/\log\log a)$ uniformly for $a \geq b \geq 1$. A logarithmic lower bound for algorithms must therefore use restrictions beyond uniformity, such as finite program syntax or determinism.

6 An almost-all lower bound

The difficult pairs of [6, Theorem 6C.5] give $\Omega(\log\log a)$ on a sparse set. An elementary Diophantine argument gives the same order for almost all coprime pairs, simultaneously for every derivation.

Proposition 9. *For integers $d \geq 1$ and $N \geq 3$, at most $40(2+\ln N)N^{2-2^{-d}}$ coprime pairs $2 \leq b < a \leq N$ satisfy $D(a,b) \leq d$. Hence $D(a,b) \geq (1-o(1))\log\log N$ for all but $o(N^2)$ of these pairs.*

Proof. For $B \geq 1$ call (a,b) *B-regular* if $|qb - pa| \geq a/(Bq)$ whenever $q \in \mathbb{N}$, $q \geq 1$, $p \in \mathbb{Z}$ and $qb \neq pa$. We first bound the cost on regular pairs, then count the irregular ones.

Every generated value has an integer representation $v = ua + wb$. On a regular pair, $v > 0$ implies $v \geq a/(B(|u|+|w|))$: for $w \neq 0$ use regularity with $q = |w|$, and for $w = 0$ note $v \geq a$. Let H_t bound the coefficient norm $|u|+|w|$ in one such representation of each value available after t instructions, starting with $H_0 = 1$. In an instruction $r = \mathrm{rem}(x,y)$, write $x = sy + r$. Since generated values are at most a, the quotient satisfies $s \leq a/y \leq BH_t$. Subtracting s times the representation of y from that of x gives

$$H_{t+1} \leq H_t + BH_t^2 \leq 2BH_t^2, \qquad H_t \leq (2B)^{2^t-1}.$$

Producing 1 within d instructions therefore forces

$$1 \geq \frac{a}{BH_d}, \qquad a \leq \frac{(2B)^{2^d}}{2}.$$

To count irregular pairs below N, choose a witness $0 < |qb - pa| < a/(Bq)$. Dividing p, q by their gcd preserves this inequality, so assume $\gcd(p,q) = 1$. The nonzero integer $r = qb - pa$ then gives $q < a/B \leq N/B$ and $0 \leq p \leq q$. For each q, there are at most $2q$ choices of p and $2N/(Bq)$ choices of the signed integer r. Given p, q, r, the equation $pa \equiv -r \pmod q$ determines one residue class for a, leaving at most $N/q + 1$ choices; b is then determined. With $Q = \lfloor N/B \rfloor$, the number of irregular pairs is at most

$$\sum_{q=1}^{Q}(2q)\frac{2N}{Bq}\left(\frac{N}{q}+1\right) = \frac{4N^2}{B}\sum_{q=1}^{Q}\frac{1}{q} + \frac{4NQ}{B}$$
$$\leq \frac{4N^2}{B}(2+\ln N),$$

using $Q \leq N$, $B \geq 1$, and the harmonic-sum bound; an empty sum is zero.

Now split into dyadic ranges $X/2 < a \leq X$, and take $B = X^{2^{-d}}/2$. If $B \geq 1$, a regular pair with $D \leq d$ would have $a \leq X/2$, so all such pairs in the range are irregular. Their number is at most $8(2+\ln X)X^{2-2^{-d}}$. When $B < 1$, the same bound follows from the trivial bound X^2, since

$X^{2^{-d}} < 2$. Put $s = 2 - 2^{-d} \geq 3/2$. Summing over the nonempty ranges with $X = N/2^j \geq 3$, and bounding their logarithmic factors by $2 + \ln N$, gives

$$8(2+\ln N)N^s \sum_{j\geq 0} 2^{-js} = \frac{8}{1-2^{-s}}(2+\ln N)N^s \leq 40(2+\ln N)N^s.$$

For the second assertion, take $d = \lfloor \log_2 \ln N - 2\log_2 \ln\ln N \rfloor$ for sufficiently large N. Then $N^{-2^{-d}} \leq \exp(-(\ln\ln N)^2)$, so the exceptional set has size $o(N^2)$, while $d = (1-o(1))\log\log N$. □

The saving 2^{-d} in the exponent of $N^{2-2^{-d}}$ shrinks as d grows, so this argument does not reach the upper-bound scale $\log N/\log\log N$.

7 Fixed programs with arbitrarily small coefficients

For a nondeterministic program P, let $c_P(a,b)$ be the minimum number of remainder calls among its convergent computations. We require at least one such computation on each input and require every convergent computation to return the gcd. These are the conventions of [6, (2E-2), (2E-5)].

Theorem 10. *For each integer $p \geq 1$, there is a finite nondeterministic recursive program P_p over $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ computing the gcd such that*

$$\limsup_{a\to\infty} \max_{1\leq b\leq a} \frac{c_{P_p}(a,b)}{\log a} \leq \frac{3}{p}.$$

Its generator, defined below, has $W = 4^p + 5$ individual parameters, and no internal function has larger arity. Consequently, for every $\varepsilon > 0$ there are a fixed program P and a constant $B \geq 0$ such that

$$\forall a \geq b \geq 1, \qquad c_P(a,b) \leq \varepsilon \log a + B.$$

The program may depend on ε, but not on the inputs. The Main Conjecture allows its lower-bound coefficient to depend on the program, so this family is compatible with it. No single sublogarithmic program is obtained. Neither the coefficient nor the arity bound is claimed optimal.

The program. Fix $W \geq 3$ and write $\vec{x} = (x_0, \ldots, x_{W-1})$ and $\vec{x}[i \leftarrow E]$ for the tuple with its ith expression replaced by E. The generator can return a parameter, replace one by a remainder, run a recursive subcomputation while retaining the caller's other values, or copy parameters into new positions. These four actions are expressed by the finite alternatives

$$\begin{aligned}
\mathrm{choose}(\vec{x}) &= x_i && (0 \leq i < W), && \text{(C)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{choose}(\vec{x}), && && \text{(G1)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\vec{x}[i \leftarrow \mathrm{rem}(\mathrm{choose}(\vec{x}), \mathrm{choose}(\vec{x}))]) && (0 \leq i < W), && \text{(G2)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\vec{x}[i \leftarrow \mathrm{gen}(\vec{x})]) && (0 \leq i < W), && \text{(G3)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\mathrm{choose}(\vec{x}), \ldots, \mathrm{choose}(\vec{x})). && && \text{(G4)}
\end{aligned}$$

There are W independent selector calls in (G4). Tuple notation and indices are syntax abbreviations: after fixing W, each line expands into ordinary first-order terms with fixed arities. No tuple, index, numeric constant, or update operation is supplied as runtime data or a primitive. The nested generator in (G3) is evaluated once; its caller's other parameters remain available when it returns. The recursion stack can grow, so W is an arity bound, not a total storage bound.

Add $\mathrm{loop}(z) = \mathrm{loop}(z)$ and define $\mathrm{accept}(a, b, g)$ by nested zero-test conditionals: if $g = 0$, loop; otherwise test $\mathrm{rem}(a, g) = 0$ and then $\mathrm{rem}(b, g) = 0$, returning g if both tests succeed and looping otherwise. The head is

$$P_W(a, b) = \text{if } \mathrm{eq}_0(b) \text{ then } a \text{ else } \mathrm{accept}(a, b, \mathrm{gen}(a, b, a, \ldots, a)).$$

The four internal symbols have arities $W, W, 3, 1$ (for choose, gen, accept, loop). Every common divisor of the generator's parameters divides its output, by induction on a finite evaluation. Thus a candidate is divisible by $\gcd(a, b)$; successful verification proves the reverse divisibility. Every convergent computation therefore returns the gcd. There is always a convergent computation following Euclid, using parameter selection and copying to retain the current pair. For $b > 0$, final verification costs exactly two remainder calls.

This syntax is a finite recursive program in the sense of [6, §§2A, 2E]; Appendix A records the Lean check that its evaluation trees correspond, with equal remainder cost, to convergent computations of a specialization of the book's recursive machine.

Proof of Theorem 10. We follow the construction of Section 4: store the prefix, contract, then recurse twice through nested generator calls. Take $P_p = P_W$ with $W = 4^p + 5$; that is, set $k = 2^p$, $h = k^2+1$ (the prefix length, as in Corollary 4 but now with p fixed), and $W = h+4$. Store $r_0, \ldots, r_h$ in $h+1$ parameters, reserving three parameters for the modulus and two scratch residues. Lemmas 2 and 3 give a contraction by a factor greater than k in at most three remainder calls, preserving the prefix. The program can choose precisely the operands of each contraction.

The *generator cost* of (a, b) is the minimum number of remainder calls in an evaluation of gen that returns $\gcd(a, b)$ from $(a, b, a, \ldots, a)$. Let $T_W(n)$ be its maximum over $a, b \le 2^n$. Euclid gives $T_W(n) = O(n)$. To *route* to x, y means to apply (G4) to pass to the tuple $(x, y, x, \ldots, x)$, at no remainder cost. Copying parameters gives symmetry. Multiplying every value by $g = \gcd(a, b)$ turns a run on $(a/g, b/g)$ that returns 1 into a run on (a, b) that returns g, with the same remainder count. It therefore suffices to analyze coprime inputs.

Fix $0 < \beta < 1/2$ and put $\alpha = 1 - \beta/2$. Use the early threshold N^α and the target modulus N^β, where $N = 2^n$. If no prefix remainder is at most N^α, then $r_h > N^\alpha$ and $2a/r_h < 2N^{1-\alpha} < N^\beta$ for all sufficiently large n. There are at least $k+1$ distinct prefix residues modulo every modulus used before stopping, since $h > k^2$. At most $\lceil n/p \rceil$ contractions produce a positive $m \le N^\beta$.

In the early case, route directly to the smaller consecutive Euclidean pair. Otherwise form $u = a \bmod m$ and $v = b \bmod m$ in the scratch positions. A nested generator call, with its parameters routed to copies of m, u, returns $d = \gcd(m, u)$ while its caller retains v. Replace the modulus parameter by d and route to copies of d, v for the second gcd computation. The result is $\gcd(m, a, b) = 1$. All these routes and nested calls are available in the same fixed program. Thus, eventually in n,

$$T_W(n) \le \max\left\{T_W(\lceil \alpha n \rceil) + h + 1, \quad 2T_W(\lceil \beta n \rceil) + h + 3\lceil n/p \rceil + 2\right\}.$$

Set $L_W = \limsup_{n\to\infty} T_W(n)/n$, finite and nonnegative by Euclid. Since h is fixed, division by n and passage to upper limits gives

$$L_W \le \max\{\alpha L_W,\ 2\beta L_W + 3/p\}.$$

As $\alpha < 1$ and $L_W \ge 0$, this implies $(1 - 2\beta)L_W \le 3/p$. The program is independent of β, so letting $\beta \downarrow 0$ gives $L_W \le 3/p$. Take $n = \lceil \log a \rceil$ and add the two verification calls. Finally, choose $p > 3/\varepsilon$ and absorb the finitely many smaller inputs in B. □

8 Discussion and open problems

The gap. The intrinsic complexity of coprimeness in $\mathcal{N}_\varepsilon$ is at least $\Omega(\log\log a)$ on almost all coprime pairs and at most $O(\log a/\log\log a)$ on all positive pairs. Closing this gap is open. In particular, the upper bound is not known to be optimal for remainder-only derivations, whereas Bshouty's corresponding bound is optimal in his model.

From a growing prefix to one program. The remaining obstacle in turning the construction into a sublogarithmic program is reusable access to a prefix whose length grows with the input. Selecting a close pair of residues costs nothing in a derivation, whereas computing all prefix residues would take about h calls per contraction. The programs of Section 7 select operands nondeterministically from a fixed number of parameters. Can one fixed first-order recursive program retain and repeatedly access a growing prefix at the same remainder cost? The book's Proposition 2E.1 converts each certificate into a computation of some program [6, Proposition 2E.1]; it does not supply one program for all certificates.

Stronger lower-bound methods. The limit established in Section 5 leaves room for arguments combining homomorphisms with additional restrictions on computations. In particular, the conjecture of a logarithmic lower bound for deterministic uniform processes remains open [5, Conjecture 3.9, p. 23], [6, §4H and p. 188].

9 Computational checks

The offline Python planner `remainder_only_certificate.py` in the accompanying repository `https://github.com/jasondavies/euclid-uniform-processes` uses the parameters of Section 4 and unrestricted arithmetic to choose instructions. An independent replay then checks operand availability, nonzero divisors, every remainder equation and the final gcd. Below a base size (256 bits by default) it uses Euclid's algorithm. On one pseudorandom coprime pair of each size, generated from its default seed, it gives these instruction counts:

Input bits	This construction	Euclid (to reach 1)
512	196	326
2048	669	1204
8192	2059	4754

These examples illustrate finite derivations; the asymptotic claim follows from the proof. The accompanying repository also checks scaled and noncoprime inputs, rejects invalid traces, and exhaustively tests the prefix lemma over a finite range. For Section 7, a separate compiler and interpreter replay successful choices in finite first-order programs, evaluating arguments right to left as the book's machine does. The `README.md` records commands, test ranges, seeds, and results. The Python interpreter is not formally verified; unrestricted arithmetic used to plan choices is outside the programs.

A Formal verification

The development uses Lean 4.35.0 [2] with mathlib [4] at its v4.35.0 tag; the pinned mathlib commit is `56a005855ea17e0b0958f73ec801ae140b0a1679`. The development contains no proof

placeholders or added axioms; the main-result audit reports only `propext`, `Classical.choice`, and `Quot.sound`. Table 1 gives the main declarations; the README of the repository https://github.com/jasondavies/euclid-uniform-processes gives the complete map and reproduction commands. The general theory of uniform processes, including Theorem 4F.1 used in Corollary 7, and the historical lower bounds are cited, not formalized. The certificate bounds and conjecture refutations are formalized independently of that theory.

The development is also packaged for the Palomar registry: `Challenge.lean` imports only mathlib and states the main theorems with every definition visible, and `Solution.lean` proves them. Palomar's comparator checks that the proved statements are exactly those of `Challenge.lean` and replays the proofs through Lean's kernel and independent kernels. The eleven compared theorems are marked † in Table 1.

Table 1: Main results and Lean declarations, all in the namespace `ShortGCD`. Names marked † are compared against `Challenge.lean`.

Paper	Lean declaration(s)
Theorem 1	`uniform_real_bound`, `sizeCost_isBigO`
Normalization (1)	`G_normalize`
Lemma 2	`small_remainder`
Lemma 3	`prefix_pairs_distinct`, `residue_card_sq`
Corollary 4	`uniform_real_bound_six`†, `uniform_intrinsic_bound_six`†
Proposition 5	`callsRem_eq_D`, `G_eq_D`, `callsCoprimeness_eq_callsRem`, `gcd_certificate_lower`, `callsGcd_le`, `callsRemGcd_le_callsGcd`, `G_le_callsRemGcd`†, `G_le_callsGcd`†, `callsCoprimeness_eq_G`†
Remark 6	`callsCoprimeness_le`
Corollary 7	Cited Theorem 4F.1; not formalized here
Corollary 8	`not_conjecture_3_4`†, `not_main_conjecture_4F5`†, `not_main_conjecture_4F5_Neps`†; literal versions `not_conjecture_3_4_literal`†, `not_main_conjecture_4F5_literal`†
Proposition 9	`AlmostAll.card_D_le`, `AlmostAll.tendsto_card_within_loglog`, `tendsto_card_callsCoprimeness_loglog`†
Theorem 10: cost	`RegisterProgram.maxCost_step`, `RegisterProgram.Fixed.limsup_profile_le`
Theorem 10: syntax	`RegisterProgram.Syntax.eval_head_correct`, `RegisterProgram.Syntax.eval_head_exists`, `RegisterProgram.Syntax.arbitrarily_small_program`
Operational correspondence	`RegisterProgram.Syntax.Machine.eval_iff_steps`, `RegisterProgram.Syntax.Machine.arbitrarily_small_program`

Arithmetic bounds. Costs are lengths of explicit remainder derivations. The finite theorem `sizeCost_step` proves the recurrence from natural-number size parameters; it does not assume a recurrence for the cost function. For the initial Big-O proof, Lean uses $j = \lfloor \log_2 n \rfloor$, $p = \lfloor j/4 \rfloor$ and $h = 2^{2p} + 1$ in place of $h = \lceil \sqrt{n} \rceil$, and checks the integer rounding estimates before passing to real logarithms. The refinement uses the tunable parameters of Corollary 4 and checks the rounding limits preserving coefficient 6. For Proposition 9, Lean takes $B = a^{2^{-d}}/2$ separately for each pair instead of splitting into dyadic ranges. It proves the stated constant 40 and the exceptional-set limit for every fixed $\varepsilon > 0$, with threshold $(1 - \varepsilon) \log_2 \log_2 N$.

Programs and machine computations. Generator runs translate into evaluations of finite first-order syntax with exactly the same remainder count, including final verification. Every terminating evaluation is correct, and a successful evaluation exists on every input. Lean also proves exact-cost equivalence with a specialization of the book's two-stack machine: arguments are evaluated right to left, zero tests and branches are separate transitions, and only remainder transitions are charged [6, Table 1, pp. 75–76]. The forward proof embeds each argument evaluation in its calling context; the reverse proof decomposes a convergent computation into its argument and body evaluations. Closures represent substitution in machine states, not extra program data. This checks the language and program family used here; the expanded note `research/nondeterministic-prefix.tex` gives the operational details and the Python compiler and interpreter.

References

[1] Nader H. Bshouty. *Euclidean GCD algorithm is not optimal*. Research Report 1989-376-38, Department of Computer Science, University of Calgary, December 1989. doi:10.11575/PRISM/30475.

[2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, vol. 12699, pp. 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.

[3] Donald E. Knuth. *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, Reading, MA, 1997.

[4] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pp. 367–381. ACM, 2020. doi:10.1145/3372885.3373824.

[5] Yiannis N. Moschovakis. *Absolute lower bounds for uniform processes*. Unpublished lecture notes (first draft), January 25, 2012. `https://www.math.ucla.edu/~ynm/nadalb.pdf`.

[6] Yiannis N. Moschovakis. *Abstract Recursion and Intrinsic Complexity*. Lecture Notes in Logic, vol. 48. Cambridge University Press and Association for Symbolic Logic, 2019. doi:10.1017/9781108234238.

[7] J. Stein. Computational problems associated with Racah algebra. *Journal of Computational Physics* 1(3):397–405, 1967. doi:10.1016/0021-9991(67)90047-2.

[8] Lou van den Dries and Yiannis N. Moschovakis. Is the Euclidean algorithm optimal among its peers? *Bulletin of Symbolic Logic* 10(3):390–418, 2004. doi:10.2178/bsl/1102022663.

[9] Lou van den Dries and Yiannis N. Moschovakis. Arithmetic complexity. *ACM Transactions on Computational Logic* 10(1), Article 2, 49 pp., 2009. doi:10.1145/1459010.1459012.